Hands-on Project: C++ Implementation of the Parareal Algorithm for ODEs

Master's Course in Scientific Computing

October 28, 2025

1 Context and motivation

The standard algorithms for the integration of a system of ordinary differential equations (ODEs) of the form

$$\frac{du}{dt} = f(t, u), \quad u(0) = u_0,$$

are intrinsically serial, since the computation of the approximate solution u_n at time t_n can be performed only after computing the solution u_{n-1} at the previous time step. Nonetheless, some study of parallel techniques for ODEs has been carried out in recent years, and an interesting algorithm is Parareal, originally introduced in [1].

Parareal combines a cheap coarse integrator with an accurate fine integrator, overlapping their execution across time slices to attain speedup over a purely serial fine integration [1, 2, 8]. An accessible overview can be found on the dedicated topic page on Wikipedia [7].

Parareal algorithm: formulation

Consider an initial value problem (IVP) on $[0, T]$:

$$\frac{du}{dt}(t) = f(t, u(t)),\ t \in (0, T], \quad u(0) = u_0,$$

with $u(t) \in \mathbb{R}^d$, $d \geq 1$ and $f : \mathbb{R} \times \mathbb{R}^d \to \mathbb{R}^d$ sufficiently regular. If $d = 1$ the problem is said to be scalar, if $d > 1$ we have a *system of ordinary differential equations* (ODE). We recall that the the special case in which $f(t, u) = f(u)$ is called "autonomous".

Usual methods for ODEs advance in time and compute a succession $u_n \approx u(t_n)$ of approximations at time steps t_n, where the computation u_n requires the knowledge at u_{n-1}. Therefore, they are intrinsically not parallelizable.

The parareal method tries to overcome this limitation bu adopting a *multiple shooting* strategy. It is based, like the other methods, on the partition of $[0, T]$ into N subintervals $[T_n, T_{n+1}]$ with $T_0 = 0$, $T_N = T$, and the use of two *propagators*:

- A coarse propagator G: less accurate but cheap, that operates with a $\Delta t = T_{n+1} - T_n$. The coarse propagator uses a standard advancing scheme, typically a Runge Kutta scheme, and is essentially a scalar step.
- A fine propagator F: accurate but more expensive than G, applied on each slice $[T_n, T_{n+1}]$, Either using a solver of higher order than G or using M fine time steps of size $\delta t = \Delta t/M$ on each slice (sometimes both strategies are implemented).

More precisely, we indicate with $G(T_n, T_{n+1}, u_n)$ the procedure that, given u_n, computes $u_{n+1} \approx u(T_{n+1})$ with a cheap solver, and with $F(T_n, T_{n+1}, u_n)$ the procedure that computes $\tilde{u}_{n+1}$ considering the problem

$$\frac{d\tilde{u}}{dt} = f(t, \tilde{u}),\ t \in (T_n, T_{n+1}], \quad \tilde{u}(T_n) = u_n,$$

with an expensive but accurate method.

In the the case F also computes the solution at intermediate times for each coarse time slab, the final solution is obtained by concatenating the accurate solver solutions in each time slab. We give some further details

Initialization (iteration 0): Initial serial coarse sweep.
Compute

$$u_0^{(0)} = u_0, \qquad u_{n+1}^{(0)} = G(T_n, T_{n+1}, u_n^{(0)}), \quad n = 0, \ldots, N-1.$$

Parareal iterations: For $k = 0, 1, \ldots$:

- In parallel, for all slices $[T_n, T_{n+1}]$, solve

$$\tilde{u}_{n+1}^{(k+1)} = F(T_n, T_{n+1}, u_n^{(k)}), \quad n = 0, \ldots, N-1.$$

- Followed by a serial coarse correction,

$$u_{n+1}^{(k+1)} = G(T_n, T_{n+1}, u_n^{(k+1)}) + \tilde{u}_{n+1}^{(k+1)} - G(T_n, T_{n+1}, u_n^{(k)}), \quad n = 0, \ldots, N-1.$$

The iteration is stopped when a stopping criterion is met, typicaly

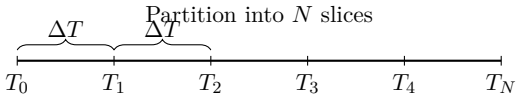
$$\max_n \|u_n^{(k+1)} - u_n^{(k)}\| \leq \varepsilon.$$

Note that when F also computes approximations at intermediate times at each coarse time slab, the final solution is obtained by concatenating the accurate solver solutions in each time slab.

Under ideal conditions, Parareal reproduces the fine propagator solution in at most N iterations. However, to achieve speedup, one needs $K \ll N$ [2], being K the number of iterations performed to reach the desired tolerance.

TikZ diagrams: Parareal workflow (compact)

Time partitioning into slices

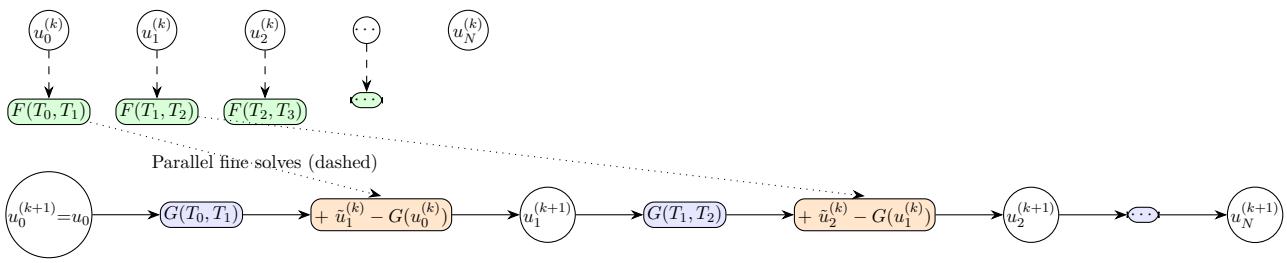


Initialization: coarse serial sweep



Coarse propagation (serial)

Parareal iteration: parallel fine + serial correction



Project overview

Team: 5 students.

1. Choose and implement the methods chosen for the G and F. the simplest choice is to use a single Runge-Kutta embedded scheme, for instance the Runge-Kutta-Fehlberg Runge-Kutta-Fehlberg, described in the linked wikipedia page or any good book on numerical methods. If you do so, is to use it sequentially on each time slice with a low error tolerance for G and a stricter error tolerance for F. You may also decide to use a lower order embedded RK scheme (with lower number of stages) for G. For a general construction of a RK embedded scheme you may also look at the folder `RKFSolver`, where you the description and code for various embedded RK schemes, including implicit (ESKIRK) ones. The implicit schemes are more robust but require to solve a (in general) non-linear system. If you go implicit for G, you may also implement just an implicit Euler:

   $$u_{n+1} = \Delta t f(t_{n+1}, un + 1) + u_n.$$

   To solve the implicit step, you can reformulate it as finding a zero of the function (ore the system of functions) $H(u) = u - \Delta t f(t_{n+1}, u) - u_n$ and use the code in the `NSSolver` folder.
2. Implement a C++ Parareal framework for systems of ODEs:
   - Interfaces for *coarse* and *fine* propagators.
   - Partition the time interval into N time slices.
   - Execute fine solves in parallel per slice, using OpenMP; perform coarse correction serially.
3. Execute tests on linear and nonlinear ODEs:
   - Include stable problems (parabolic/stiff) and challenging cases (imaginary eigenvalues, hyperbolic behavior).
   - Assess accuracy, iterations K, speedup, and efficiency.
4. Deliverables:
   - Working C++ code with build system.
   - Short report (6–8 pages).
   - Presentation (a few slides).

Parareal pseudocode

```
// Input: f, T0, T, N (slices), u0, propagators F, G, tol, Kmax
// Output: {u_n} approx of u(T_n)
Partition [T0, T] into {T_n}_{n=0}^N with DeltaT = (T - T0)/N
// Zeroth iteration: serial coarse sweep
u[0] = u0
for n = 0..N-1:
  G.step(T_n, T_{n+1}, u[n], u[n+1])

for k = 0..Kmax-1:
  // Fine solves in parallel
  for n = 0..N-1 in parallel:
    F.step(T_n, T_{n+1}, u[n], uFine[n])
  // Coarse correction in serial
  uNew[0] = u0
  for n = 0..N-1:
    std::vector<double> Gu_k, Gu_kp1;
    G.step(T_n, T_{n+1}, u[n], Gu_k);
    G.step(T_n, T_{n+1}, uNew[n], Gu_kp1);
    uNew[n+1] = Gu_kp1 + (uFine[n] - Gu_k);
  if max_n ||uNew[n] - u[n]|| <= tol: break
  u = uNew
```

Suggested test cases

Implement and discuss at least three cases.

1) Stable scalar linear ODE

$$u'(t) = \lambda u(t), \quad \lambda = -1, \quad u(0) = 1, \quad t \in [0, 10].$$

Exact solution: $u(t) = e^{-t}$, useful for absolute/relative errors. Expected: fast convergence with small K and good efficiency. Vary N (e.g., 8, 16, 32), h_f, h_c to study accuracy/cost tradeoffs [2].

2) Harmonic oscillator (imaginary eigenvalues)

$$\begin{cases} x'(t) = \omega y(t), \\ y'(t) = -\omega x(t), \end{cases} \qquad \omega = 10, \quad (x(0), y(0)) = (1, 0), \quad t \in [0, 10].$$

Hamiltonian system with purely imaginary eigenvalues. Vanilla Parareal may exhibit instability or very slow convergence (compromising speedup). Compare:

3) Nonlinear system: Van der Pol

$$\begin{cases} x'(t) = y(t), \\ y'(t) = \mu(1 - x(t)^2)\, y(t) - x(t), \end{cases} \qquad \mu = 5, \quad (x(0), y(0)) = (2, 0), \quad t \in [0, 40].$$

Moderately stiff. Assess:

- Robust G (implicit Euler or explicit with smaller step) and accurate F (RK4).

4 More challenging!

Use parareal method to speedup the time evolution of a parabolic problem of the type

$$\begin{cases} \frac{\partial y}{\partial t} - \mu \triangle y = f(t, x), & x \in \Omega(0, 1)^2 \subset \mathbb{R}^2,\, t \in (0, T) \\ y(0, x) = 0 & x \in \partial\Omega\, t = 0 \\ y(x, 0) = g(x) & x \in \Omega\, t \in (0, T) \end{cases}$$

after having discretised it in space with finite differences or finite elements.

Metrics and reporting

For each case:

- Convergence in k: $\max_n \|u_n^{(k+1)} - u_n^{(k)}\|$.
- Final error vs reference (analytic or serial fine integration).
- Total runtime, speedup $S = T_{\text{serial-fine}}/T_{\text{parareal}}$, efficiency $E = S/P$ (P = number of threads).
- Dependence of the results on N.
- Discussion on efficiency with respect different class of problems or problem parameter values.

Suggested work plan (5 students)

- Kick-off, assign roles (Lead dev, Dev 2, Dev 3, Analyst, Writer/Presenter).
- C++ skeleton (interfaces, vector utilities, RK4 and Euler/Heun).
- Parareal driver + parallelization (OpenMP) + unit tests.
- Case 1 complete (validation, timings, plots).
- Cases 2 and 3, tuning G and F, study K vs N.
- Profiling, speedup/efficiency analysis, stability discussion.
- Report writing, slide preparation, dry-run presentation.

Optional extensions (extra)

- **Krylov-subspace enhanced Parareal:** enrich G by projecting onto the span of accumulated fine data to accelerate convergence and improve stability for hyperbolic problems [3].
- **Parareal–SDC/PFASST hybrids:** replace F and G with SDC and assess the impact on parallel efficiency [4, 5].
- **MGRIT/XBraid:** conceptual comparison with multilevel multigrid-in-time [6].

Core bibliography

References

[1] J.-L. Lions, Y. Maday, G. Turinici, "A 'parareal' in time discretization of PDE's", *C. R. Acad. Sci. Paris*, 332(7):661–668, 2001. HAL preprint available.

[2] M. J. Gander, S. Vandewalle, "Analysis of the Parareal Time-Parallel Time-Integration Method", *SIAM J. Sci. Comput.*, 29(2):556–578, 2007.

[3] M. Gander, M. Petcu, "Analysis of a Krylov subspace enhanced parareal algorithm for linear problems", *ESAIM: Proceedings*, 25:114–129, 2008.

[4] M. L. Minion, "A Hybrid Parareal Spectral Deferred Corrections Method", *CAMCoS*, 5(2):265–301, 2010.

[5] M. Emmett, M. Minion, "Toward an efficient parallel in time method for PDEs", *CAMCoS*, 7(1):105–132, 2012.

[6] R. Falgout et al., "Parallel Time Integration with Multigrid", *SIAM J. Sci. Comput.*, 36(6):C635–C661, 2014.

[7] Parareal, Wikipedia entry (overview, history, variants).

[8] Parallel-in-Time community: references and resources, https://www.parallel-in-time.org/.

Appendix: Minimal implementation excerpt (illustrative)

Note: this is indicative; the project should include a more modular code, with classes, and tests. Moreover, it has been produced by GPT. So it some detail con be incorrect and the C++ style not the best possible.

```
#include <vector>
#include <functional>
#include <cmath>
#include <omp.h>

using Vec = std::vector<double>;
using RHS = std::function<void(double,const Vec&,Vec&)>;

struct Propagator {
 virtual ~Propagator() = default;
 virtual void step(double t0, double t1, const Vec& u0, Vec& u1) const = 0;
};

struct RK4 : Propagator {
 RHS f; double h;
```

```
 RK4(RHS f_, double h_) : f(f_), h(h_) {}
 void step(double t0, double t1, const Vec& u0, Vec& u1) const override {
  double t = t0; Vec u = u0, k1(u.size()), k2(u.size()), k3(u.size()), k4(u.size()), tmp(u.s
  auto axpy = [](Vec& y, double a, const Vec& x){ for(size_t i=0;i<y.size();++i) y[i]+=a*x[i
  while (t < t1 - 1e-14) {
   double hh = std::min(h, t1 - t);
   f(t, u, k1);
   tmp = u; axpy(tmp, 0.5*hh, k1); f(t+0.5*hh, tmp, k2);
   tmp = u; axpy(tmp, 0.5*hh, k2); f(t+0.5*hh, tmp, k3);
   tmp = u; axpy(tmp, hh,      k3); f(t+hh,      tmp, k4);
   for (size_t i=0;i<u.size();++i)
   u[i] += (hh/6.0)*(k1[i]+2*k2[i]+2*k3[i]+k4[i]);
   t += hh;
  }
  u1 = u;
 }
};

struct Euler : Propagator {
 RHS f; double h;
 Euler(RHS f_, double h_) : f(f_), h(h_) {}
 void step(double t0, double t1, const Vec& u0, Vec& u1) const override {
  double t = t0; Vec u = u0, k(u.size());
  auto axpy = [](Vec& y, double a, const Vec& x){ for(size_t i=0;i<y.size();++i) y[i]+=a*x[i
  while (t < t1 - 1e-14) {
   double hh = std::min(h, t1 - t);
   f(t, u, k);
   Vec du = k; for(size_t i=0;i<du.size();++i) du[i] *= hh;
   axpy(u, 1.0, du);
   t += hh;
  }
  u1 = u;
 }
};

struct Parareal {
 const Propagator &F, &G;
 int N, Kmax; double tol;
 Parareal(const Propagator& F_, const Propagator& G_, int N_, int Kmax_, double tol_)
 : F(F_), G(G_), N(N_), Kmax(Kmax_), tol(tol_) {}
 void solve(double T0, double T1, const Vec& u0, std::vector<Vec>& U) const {
  U.assign(N+1, u0); std::vector<double> T(N+1);
  double DT = (T1 - T0)/N; for(int n=0;n<=N;++n) T[n]=T0+n*DT;
  // Zeroth iteration
  for (int n=0; n<N; ++n) G.step(T[n], T[n+1], U[n], U[n+1]);
  std::vector<Vec> UF(N), Unew(N+1, u0), Gu_k(N), Gu_kp1(N);
  auto norminf = [](const Vec& a,const Vec& b){ double m=0; for(size_t i=0;i<a.size();++i) m
  for (int k=0; k<Kmax; ++k) {
   // Fine in parallel
   #pragma omp parallel for
   for (int n=0; n<N; ++n) F.step(T[n], T[n+1], U[n], UF[n]);
   Unew[0] = u0;
   for (int n=0; n<N; ++n) {
```

```
    Vec tmp;
    G.step(T[n], T[n+1], U[n], tmp);       Gu_k[n] = tmp;
    G.step(T[n], T[n+1], Unew[n], tmp);    Gu_kp1[n] = tmp;
    Vec corr = UF[n];
    for (size_t i=0;i<corr.size();++i) corr[i] -= Gu_k[n][i];
    Vec unp1 = Gu_kp1[n];
    for (size_t i=0;i<unp1.size();++i) unp1[i] += corr[i];
    Unew[n+1] = unp1;
   }
   double diff = 0.0;
   for (int n=0; n<=N; ++n) diff = std::max(diff, norminf(Unew[n], U[n]));
   U = Unew;
   if (diff <= tol) break;
  }
 }
};
```